

Compilation

Analyse lexicale

Nicolas Delestre

Rappels 1 / 2

But de l'analyse lexicale

- Passer d'un flot de caractères en flot de couples $\langle \text{unité lexicale}, \text{valeur} \rangle$ (valeur optionnelle), par exemple, passer de :

"position:=initiale+vitesse*60"
à

$\langle \text{id}, "position" \rangle \langle \text{aff}, \rangle \langle \text{id}, "initiale" \rangle \langle \text{add}, \rangle \langle \text{id}, "vitesse" \rangle \langle \text{mul}, \rangle \langle \text{nb}, 60 \rangle$

Comment ?

- En reconnaissant les lexèmes (lexème = plus longue suite de caractères permettant d'identifier une unité lexicale)
- Chaque unité lexicale forme un langage formel, et les lexèmes les mots de ce langage
 - Certains lexèmes sont connus à l'avance, ils peuvent être listés
 - Certains peuvent être représentés à l'aide de motif (comme par exemple les nombres)
- Ces caractéristiques définissent le type de ces langages : langages rationnels (type 3)

Des grammaires linéaires à gauche (ou à droite)

- Les grammaires formelles linéaires à gauche (ou à droite) permettent de dénoter un langage rationnel :
 - Les symboles terminaux (l'alphabet) sont des caractères (historiquement ASCII, mais aujourd'hui pour certains langages de programmation UTF-8)
 - les règles de production de la forme $A \rightarrow aB \mid \epsilon$
- Mais ces règles de production ne sont pas faciles à créer (alors que ces langages sont relativement simples)
- Il existe un autre formalisme plus adapté, équivalent aux grammaires linéaires à gauche (ou à droite) : les *expressions rationnelles*

Définition

- Une expression rationnelle dénote un langage rationnel : elle permet d'exprimer à l'aide d'une seule expression l'ensemble (fini ou infini) des mots d'un langage rationnel

Construction d'une expression rationnelle

- Le mot vide ϵ est une expression rationnelle
- Les lettres de l'alphabet sont des expressions rationnelles
- Si e est une expression rationnelle, alors (e) est une expression rationnelle
- Les opérations suivantes permettent de construire de nouvelles expressions rationnelles :
 - Opération binaire de concaténation, opérateur $.$, souvent non marqué : $e_1.e_2$ est noté e_1e_2
 - Opération binaire d'union, opérateur $|$, $e_1|e_2$
 - Opération unaire de fermeture de Kleene, de 0 à $+\infty$ concaténations, opérateur $*$, e^*

Exemples pour l'alphabet $\Sigma = \{0, 1\}$

- Les naturels en binaire : $0|(1(0|1)^*)$
- Les naturels paires en binaire : $0|(1(0|1)^*0)$

Exemple pour l'alphabet composé des caractères ASCII

- Les naturels en décimal : $0|((1|2|3|4|5|6|7|8|9)(0|1|2|3|4|5|6|7|8|9)^*)$

Extensions

- Opérations (e est une expression rationnelle) :
 - Opération unaire d'addition : $e^+ = ee^*$
 - Opération unaire « optionnel » : $e? = e|\epsilon$
 - Opération binaire puissance : $e^1 = e$ et $e^n = ee^{n-1}$
- Regroupement ($a, b, \dots$ des lettres qui se suivent dans l'alphabet) :
 - $[abc] = (a|b|c)$
 - $[a - d] = (a|b|c|d)$

Exemples pour l'alphabet composé des caractères ASCII

- Les naturels en décimal (écriture « humaine ») : $0|([1 - 9][0 - 9]^*)$
- Les réels en décimal (écriture « humaine ») : $0|(-?[1 - 9][0 - 9]^*.[0 - 9]^+)$
- Les identifiants du langage pascal : $[_a - zA - Z][_a - zA - Z0 - 9]^*$

Définition rationnelle

Constats

- Rapidement la lecture d'une expression rationnelle peut être difficile
- Les définitions rationnelles permettent d'associer un identifiant à une expression rationnelle
- Une expression rationnelle peut utiliser les définitions rationnelles déjà définies (pas celles qui ne sont pas encore définies)
- Notation : *identifiant* $\rightarrow$ *expression*

Exemple les constantes « nombre » en Pascal

ENS^a $\rightarrow [0 - 9]^+$

RNS^b $\rightarrow \mathbf{ENS.ENR|ENS.ENR} [eE] (\mathbf{ENS|} - \mathbf{ENS})|\mathbf{ENS} [eE] (\mathbf{ENS|} - \mathbf{ENS})$

Nombre $\rightarrow \mathbf{ENS|RNS|} - \mathbf{ENS|} - \mathbf{RNS}$

a. Entier Non Signe

b. Reel Non Signe

Problème de la reconnaissance

Rappels sur l'objectifs de la théorie des langages

- Être capable de représenter un langage
- Être capable de savoir si un mot appartient (ou pas) à un langage et quelle est la complexité de l'algorithme pour le vérifier

Constats

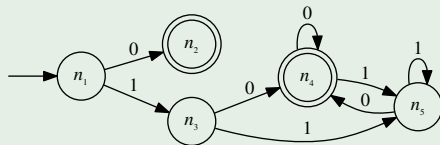
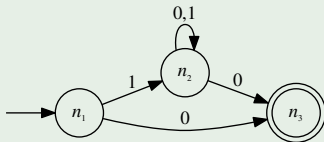
- Alors que les grammaires formelles permettent aussi de vérifier si un mot appartient au langage engendré, grâce au mécanisme de dérivation, les expressions rationnelles ne le permettent pas
- Mais il existe un troisième « outil » permettant de représenter des langages rationnels : *les automates à états finis*

Définition

- Un automate à états finis est un graphe orienté étiqueté et valué tel que :
 - Les nœuds du graphe représentent les états de l'automate :
 - L'un des nœuds représente l'état initial (identifié par un arc entrant sans source)
 - Un ou plusieurs nœuds représentent les états finaux (identifiés par un double cercle)
 - La valeur d'un arc est soit ϵ ou soit une lettre de l'alphabet
- Lorsque pour chaque nœud il n'y a pas deux arcs sortants valués par la même lettre, l'automate est déterministe (dans le cas contraire c'est un automate non déterministe)
- Lorsque pour chaque nœud il n'y a aucun arc étiqueté par ϵ , l'automate est sans ϵ -transitions (dans le cas contraire c'est un automate avec ϵ -transitions)
- Deux automates sont équivalents s'ils reconnaissent les mots d'un même langage

Nombres pairs en binaire

- Automate non déterministe :
- Automate déterministe :



Exercice

- La suite de symboles 11010 représente-t-elle un nombre pair ?

Automate à états finis 3 / 4

Algorithme de vérification d'appartenance d'une suite de symboles $ls = l_1 l_2 \dots l_n$ à un langage rationnel dénoté par un automate à états finis déterministe

fonction estUnMotDuLangage ($ls : \text{Liste} \langle \text{Symbole} \rangle$, $a : \text{AutomateAEtatsFinisDeterministe}$) : **Booleen**

Déclaration $i : \text{NaturelNonNul}$
 $e : \text{Etat}$

debut

$i \leftarrow 1$

$e \leftarrow \text{etatInitial}(a)$

tant que $i \leq \text{longueur}(ls)$ et $\text{existeArcSortant}(a, e, \text{obtenirElement}(ls, i))$ **faire**

$e \leftarrow \text{etatSuivant}(a, e, \text{obtenirElement}(ls, i))$

$i \leftarrow i + 1$

fintantque

retourner $(i > \text{longueur}(m))$ et $\text{estUnEtatFinal}(a, e)$

fin

Théorèmes

- Il existe un algorithme qui permet de calculer pour toute expression rationnelle dénotant un langage L un automate avec ϵ -transitions reconnaissant les mots de L
- Il existe un algorithme qui permet de calculer pour tout automate avec ϵ -transitions un automate sans ϵ -transitions équivalent
- Il existe un algorithme qui permet de calculer pour tout automate non déterministe un automate déterministe équivalent

Corollaire

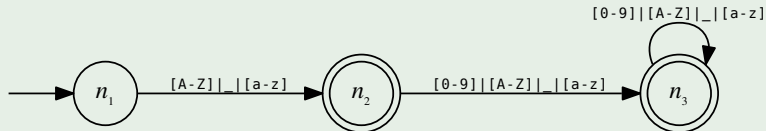
- Il existe un algorithme qui permet de calculer pour toute expression rationnelle dénotant un langage L un automate déterministe sans ϵ -transitions reconnaissant les mots de L
- <http://hackingoff.com/compilers/regular-expression-to-nfa-dfa>
<https://github.com/gnidan/solregex>

- Un langage rationnel L peut être représenté à l'aide :
 - d'une grammaire formelle linéaire à gauche (ou à droite) : pas pratique à construire
 - d'une expression rationnelle : assez facile à construire mais elles ne permettent pas de vérifier l'appartenance d'un mot m à L
 - d'un automate à états finis : pas pratique à construire mais très performant pour vérifier l'appartenance d'un mot m à L (dans le cas des automates déterministes sans ϵ -transitions)
- Il existe différents algorithmes qui quand ils sont exécutés les uns à la suite des autres permettent de transformer une expression rationnelle en un automate déterministe sans ϵ -transitions
- Une unité lexicale est un langage rationnel, elle peut être représentée par un automate déterministe
- L'ensemble des unités lexicales peut être représenté par un automate déterministe ou les états finaux indiquent l'unité lexicale reconnue

Exemple : `position:=initiale+vitesse*60` 1 / 6

Unité lexicale *id*

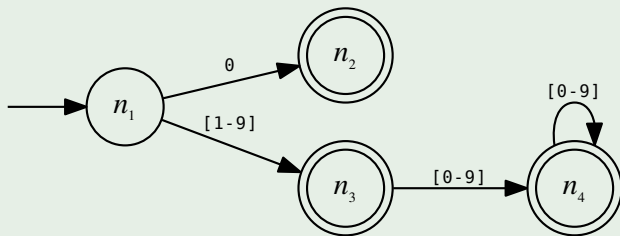
- Expression rationnelle : $[a - zA - Z_][a - zA - Z_0 - 9]^*$
- Automate déterministe :



Exemple : `position:=initiale+vitesse*60` 2 / 6

Unité lexicale *nb*

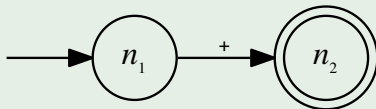
- Expression rationnelle : $0|([1-9][0-9]^*)$
- Automate déterministe :



Exemple : $\text{position} := \text{initiale} + \text{vitesse} * 60$ 3 / 6

Unité lexicale *add*

- Expression rationnelle : $+$

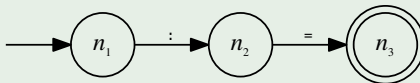


- idem pour les unités lexicales *mul*, *sous*, et *div*

Exemple : $\text{position} := \text{initiale} + \text{vitesse} * 60$ 4 / 6

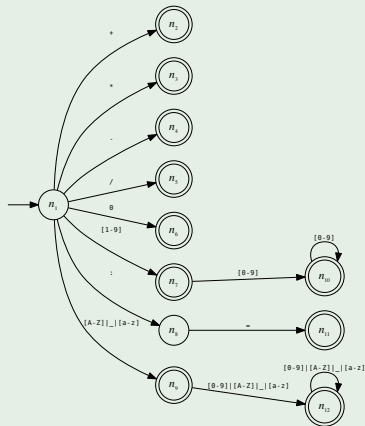
Unité lexicale *aff*

- Expression rationnelle : $:=$



Exemple : $\text{position} := \text{initiale} + \text{vitesse} * 60$ 5 / 6

Automate général



Avec les états :

- l'état n_2 (respectivement n_3, n_4 et n_5) permet de reconnaître l'unité lexicale *add* (respectivement *mul, sous, div*)
- les états n_6, n_7 et n_{10} qui permettent de reconnaître l'unité lexicale *nb*
- l'état n_{11} qui permet de reconnaître l'unité lexicale *aff*
- les états n_9 et n_{12} qui permettent de reconnaître l'unité lexicale *id*

Exemple : $position := initiale + vitesse * 60$ 6 / 6

Analyseur lexical (sans calcul des valeurs)

procédure analyseurLexical (**E** ch : Chaîne de caracteres, a : AutomateAEtatsFinisDeterministe, **S** resultat : Liste<UniteLexicale>, erreur : **Booleen**)

Déclaration i : NaturelNonNul, e : Etat

debut

 resultat $\leftarrow$ liste()

 i $\leftarrow$ 1

 e $\leftarrow$ etatInitial(a)

 erreur $\leftarrow$ FAUX

tant que i $\leq$ longueur(ch) et non erreur **faire**

si existeArcSortant(a,e,iemeCaractere(ch,i)) **alors**

 e $\leftarrow$ etatSuivant(a,e,iemeCaractere(ch,i))

 i $\leftarrow$ i+1

sinon

si estUnEtatFinal(a,e) **alors**

 inserer(resultat,longueur(resultat)+1,uniteLexicale(a,e))

 e $\leftarrow$ etatInitial(a)

sinon

 erreur $\leftarrow$ Vrai

finsi

finsi

fintantque

si non erreur et estUnEtatFinal(a,e) **alors**

 inserer(resultat,longueur(resultat),uniteLexicale(a,e))

sinon

 erreur $\leftarrow$ Vrai

finsi

fin

Conclusion

- Dans ce cours nous avons vu que :
 - les unités lexicales sont représentées par des expressions (ou définitions) rationnelles
 - à toute expression (définition) rationnelle peut être associée un automate à état fini déterministe
 - un analyseur lexical est l'association d'un automate et d'un algorithme qui permet d'identifier toutes les unités lexicales d'un compilateur ou d'un interpréteur

Un analyseur lexical peut être obtenu automatiquement à partir de couples (expression rationnelle, unité lexicale)